One Integer Engine, Nine Generations of Language Models: Measuring the Hardware Cost of Each Architectural Idea on an FPGA

Murat Atceken
Department, University
City, Country
email

***Abstract*—Language-model architectures have evolved through a sequence of ideas—recurrence, gating, attention, RMSNorm, rotary positions, gated linear units, grouped-query attention, QK-normalization—yet hardware studies evaluate them one model at a time, on different datasets, budgets and arithmetic. We implement nine architectures from Andrej Karpathy's educational repositories (bigram, MLP, RNN, GRU, bag-of-words, GPT-2, microgpt, Llama 2, nanochat) and a pretrained TinyStories Llama as integer-only programs (INT8 weights and activations, INT16 residual stream) on *one* microcoded engine targeting a low-cost Zynq-7020 FPGA. A compiler turns each PyTorch model into microcode and memory images; the same RTL—one bitstream—runs every model, bit-exact against a Python integer reference (2,846,072 logits compared, plus 60 randomized differential tests). Under an equal parameter budget we then measure, feature by feature, what each idea costs in cycles and FPGA resources and what it buys in quality, on a short-context (names) and a long-context (TinyStories) task.**

***Index Terms*—FPGA, integer-only inference, language models, transformers, quantization, hardware/architecture co-design**

I. Introduction

The architecture of today's language models is the result of many incremental ideas. Recurrent models replaced fixed-window MLPs [1], [2], gates made recurrence trainable [3], attention replaced recurrence [4], and the GPT-2 block [5] has since been modified by RMSNorm [6], rotary position embeddings [7], SwiGLU [8], grouped-query attention [9], squared-ReLU [10] and QK-normalization [11], as in Llama 2 [12]. Each idea was justified by quality. Their *hardware* cost has not been measured in a controlled way: accelerator papers specialize one model [13]–[15], and ablation studies of architectural modifications report quality only [16].

Andrej Karpathy's educational repositories happen to retrace this history with small, readable implementations: `makemore` [17] (bigram, MLP, RNN, GRU, bag-of-words), `minGPT`/`nanoGPT` [18], [19] (GPT-2), `microgpt` [20], `llama2.c` [21] (Llama 2, with pretrained TinyStories [22] models [23]) and `nanochat` [24]. Because they share one interface—predict the next token—they can be compared on the same data, budget and arithmetic. We use them as a controlled testbed and ask: *under a fixed parameter budget and integer-only arithmetic, what does each architectural idea cost in hardware, and what does it buy in quality?*

Contributions.

- A single microcoded integer engine (SystemVerilog) and compiler that run nine architecturally distinct language models and a pretrained TinyStories Llama 2 from one bitstream on a Zynq-7020, bit-exact against a Python integer reference (§V, §VI).
- Integer kernels for the modern operator set (LayerNorm/RMSNorm with a normalized integer square root, softmax with a $2^{-k/256}$ table, RoPE, SwiGLU, QK-norm, gated recurrence, causal averaging) with stated range guarantees (§IV).
- A controlled, equal-budget characterization: per-architecture and per-feature (a "ladder" from GPT-2 to Llama 2 and nanochat) cycles, LUT/DSP/BRAM and quality, with learning rates tuned per architecture and multiple seeds, on a short- and a long-context task, and three parameter budgets (§VII).
- An attribution of INT8 error in small language models—activations, not weights—and a hardware-transparent calibration recipe that halves it on TinyStories (§VII-E).

All code, models, generated memories and measurement scripts are open source.[1]

II. Background and related work

Integer-only inference. Integer-arithmetic-only inference with per-channel weight scales and fixed-point requantization [25]–[28] is the basis of our datapath. For transformers, I-BERT [29], FQ-ViT [30], I-ViT [31] and I-LLM [32] give integer approximations of GELU, softmax, LayerNorm and RMSNorm; hardware-oriented units include Softermax [33], NN-LUT [34] and SOLE [35]. That activations rather than weights limit 8-bit transformer accuracy, and that outliers matter, is known [36]–[38]; percentile calibration is standard practice [39], [40]. We confirm these effects at a much smaller scale and use them; we do not claim them as new.

[1] https://github.com/Muratatceken/mingpt-fpga

TABLE I
THE NINE ARCHITECTURES. "STATE": WHAT INFERENCE KEEPS ABOUT THE PAST.

Model	Source	Mixing	Norm	Position	State
Bigram	makemore	none	–	–	none
MLP	makemore	window of k tokens	–	window order	last k tokens
RNN	makemore	recurrence	–	implicit	h_t
GRU	makemore	gated recurrence	–	implicit	h_t
BoW	makemore	causal mean	–	learned	running sum
GPT-2	minGPT	MHA	LayerNorm	learned	KV cache
microgpt	gist	MHA	RMS (no gain)	learned	KV cache
Llama 2	llama2.c	GQA	RMSNorm	RoPE	KV cache
nanochat	nanochat	GQA + QK-norm	RMS (no gain)	RoPE	KV cache

Transformer and LLM accelerators. Attention accelerators (A^3 [41], SpAtten [42], ELSA [43]) and FPGA transformer designs (FTRANS [44], ViTA-like designs [45]) target one model family. DFX [13] runs GPT-2 on multiple FPGAs with an instruction set, FlightLLM [14] maps LLMs to a large FPGA, and [46] analyses spatial LLM acceleration. Closest in scale, HLSTransform [15] runs llama2.c's 110M TinyStories Llama on a VU9P with INT8 matrix products, LlamaF [47] and TeLLMe [48] target embedded FPGAs, SECDA-LLM [49] offloads matrix products of TinyLlama to a PYNQ-Z1, and an RNN language model has run on the same XC7Z020 [50]. RNN accelerators include ESE [51] and C-LSTM [52].

Programmable overlays. Instruction-driven engines that run many networks from one bitstream are a long line of work: Angel-Eye [53] (also on an XC7Z020), Brainwave [54], OPU [55], VTA [56] and, for NLP, NPE [57]. Our engine is in this tradition; its purpose is not peak throughput but a common, bit-exact substrate on which architectures can be compared.

Architecture versus cost. Narang et al. [16] ablate transformer modifications for quality; HAT [58] and [59] relate architecture to latency on general-purpose hardware. To our knowledge no prior work reports cycles and FPGA resources *per architectural innovation* at an equal parameter budget.

III. MODELS AND EXPERIMENTAL PROTOCOL

The family. Table I lists the nine architectures. Each is re-implemented from its source repository with one interface (token ids in, next-token logits out), including the source's initialization. The nanochat model is its core architecture (parameter-free RMSNorm, also after the embedding and before the head; half-split RoPE; QK-norm with scale 1.2; ReLU^2 MLP; GQA; untied head) without its training-specific extras.

The feature ladder. To isolate single ideas we use one configurable decoder (`Ladder`) and change one design choice per step (Table II): from the GPT-2 block (LayerNorm, learned positions, GELU, MHA, biases) to RMSNorm, to RoPE, to SwiGLU, to GQA, to the Llama 2 block (no biases, tied embeddings); a branch replaces SwiGLU by ReLU^2 and adds QK-norm as in nanochat. All ladder models share one initialization scheme so that only the architecture changes. At every step the MLP width is re-chosen so that the parameter count stays within $\pm 3\%$ of the budget.

TABLE II
FEATURE LADDER: ONE CHANGE PER STEP, EQUAL PARAMETER BUDGET.

Step	Change	Resulting block
L0	–	GPT-2 (LN, learned pos., GELU, MHA)
L1	LayerNorm $\rightarrow$ RMSNorm	
L2	learned positions $\rightarrow$ RoPE	
L3	GELU MLP $\rightarrow$ SwiGLU	
L4	MHA $\rightarrow$ GQA ($H_{kv} = H/2$)	
L5	no biases, tied embeddings	Llama 2 (llama2.c)
L6	L4 with ReLU^2 instead of SwiGLU	
L7	L6 + QK-norm	nanochat-like

Tasks and budgets. *Names* (short context): makemore's 32,033 names, character level ($V = 27$, $T = 16$), split with seed 3407 into 30,033 training, 1,000 validation and 1,000 test names; budgets of 12k, 50k and 200k parameters. *TinyStories* (long context): the GPT-4 TinyStories corpus [22] tokenized with the 512-token vocabulary of llama2.c's stories260K, $T = 256$; shard 0 is held out (validation and test windows start at story boundaries); budget 260k parameters, the size of the pretrained stories260K, which we also run. Transformers use head size 8 (the engine's attention lane count), four layers on names and five on TinyStories; recurrent and MLP models choose their widths by the same budget rule.

Training. AdamW ($\beta = (0.9, 0.99)$, weight decay 0.01), 500 warm-up steps, cosine decay to 1/30 of the peak, gradient clipping 1.0, batch 64. On names every model trains 20k steps; its learning rate is chosen per architecture from $\{10^{-3}, 3 \cdot 10^{-3}, 10^{-2}\}$ on the validation set, and test results are reported as mean $\pm$ standard deviation over 1 seeds. On TinyStories each model trains 3,000 steps (49M tokens) with one learning rate and one seed (compute budget).

IV. INTEGER SPECIFICATION

A compiler lowers each PyTorch model to a per-token *program* over a small set of operations (Table III). Affine parts of normalizations, the $1/\sqrt{d_h}$ attention scale and input scales are folded into the following weights, so the program contains only parameter-free normalizations. A float64 interpreter of the program reproduces the PyTorch logits of every model to within 10^{-4}, which validates the lowering. The program is then quantized; an integer interpreter (`IntRunner`) defines the hardware semantics, and the RTL must match it bit for bit.

Formats. Weights are INT8 with one scale per output row; biases INT32. Activations between operations are INT8 with one static scale per tensor, the residual stream is INT16 with a power-of-two scale, logits INT16. A linear layer accumulates INT8×INT8 products in INT32 and requantizes per output with $y = \lfloor (a\, m_0 + 2^{sh-1})/2^{sh} \rfloor$, $m_0 \in [2^{15}, 2^{16})$ [25]. Element-wise nonlinearities (GELU, tanh, σ, SiLU, ReLU, ReLU^2) are 256-entry tables indexed by the requantized INT8 pre-activation, so they are exact for the INT8 input grid.

Normalization. For a segment $x \in \mathbb{Z}^n$ (INT16 residual or INT8 vector) LayerNorm uses $Q = \sum_i (n x_i - s)^2 + E = n(n \sum_i x_i^2 - s^2) + E$ with $s = \sum_i x_i$, so one pass suffices; RMSNorm uses $Q = \sum_i x_i^2 + E$, where E is the integer image of ε. The inverse square root is computed as $\text{inv} =$

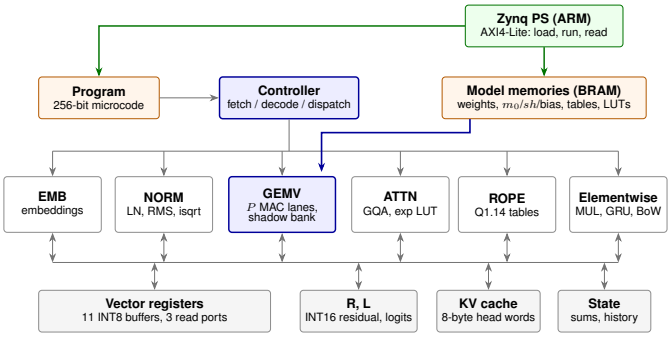


Fig. 1. The microcoded engine. A controller fetches 256-bit instructions and dispatches them to one of seven units; all model-specific content lives in memories that the Zynq PS loads at run time.

$\lfloor K2^k/\lfloor\sqrt{Q\,4^k}\rfloor\rfloor$ with $k = \max(0, \lfloor(40 - \mathrm{bitlen}(Q))/2\rfloor)$, i.e. Q is normalized to 39–40 bits before the integer square root, keeping the relative precision constant even for the short INT8 segments of QK-norm. Since $|nx_i - s| \le \sqrt{Q}$, $|(nx_i - s)\,\mathrm{inv}| \le K < 2^{40}$: the product fits 64 bits. The output scale is chosen from the calibrated range but never coarser than the guaranteed bound ($\sqrt{n-1}$ for LayerNorm, $\sqrt{n}$ for RMSNorm).

Softmax. Scores $s_j = q \cdot k_j$ are exact integers. With $u_j = (m - s_j)\,C$ ($m = \max_j s_j$, C a fixed-point constant absorbing all scales and $\log_2 e$), weights are $e_j = \mathrm{EXP}[(u_j \gg (F-8)) \,\&\, 255] \gg (u_j \gg F)$ from a table $\mathrm{EXP}[k] = \mathrm{round}(2^{15}2^{-k/256})$, so $e_j \le 2^{15}$ and $\sum_j e_j \ge 2^{15}$. One reciprocal $R = \lfloor 2^{32}/\sum_j e_j \rfloor$ per head normalizes the INT34 accumulation $\sum_j e_j v_j$; the result is a convex combination of V rows, so its scale is that of V.

Other operations. RoPE uses Q1.14 tables; the input scale is calibrated on the largest pair norm, so the rotation cannot overflow. SwiGLU multiplies two INT8 vectors and requantizes. The GRU update is $h \leftarrow h + \lfloor(z(\tilde{h} - h) + 64)/2^7\rfloor$ with $z, \tilde{h}, h$ in Q0.7. The causal mean of the bag-of-words model keeps an INT32 running sum and multiplies by a table of $\lfloor 2^{20}/(t+1)\rceil$.

Calibration. Activation scales are calibrated on 2,000 training names (100 TinyStories stories). We compare the maximum with high percentiles of $|x|$ (two passes, 4,096-bin histograms) and calibrated versus bound-derived normalization scales in §VII-E; the chosen recipe (99.99th percentile, calibrated normalization scale) is used for all models.

V. The Microcoded Engine

Instruction set. Table III lists the fifteen operations. A 256-bit instruction carries an opcode, destination and two source buffers, a mode, four 16-bit sizes, two memory base addresses and two constants (m_0/sh, the normalization constants K and E, or the softmax constant C and shift F). The compiler emits the program of a model (2 to 74 instructions) and its memory images; the controller executes the program once per token.

Units. The GEMV unit is output-stationary with P MAC lanes ($P = 8$ unless stated): it streams one P-byte weight word per cycle, accumulates P outputs in INT32, and hands them to a shadow bank so that the serial requantizer of block b overlaps the accumulation of block $b+1$. A layer of $n_{\mathrm{in}} \times n_{\mathrm{out}}$ takes $n_{\mathrm{in}}\lceil n_{\mathrm{out}}/P\rceil$ cycles; the input may be the concatenation of two buffers ($[x_t; h_{t-1}]$). The NORM unit makes one pass, normalizes Q, takes a 28-cycle integer square root and one division. The attention unit has 8 lanes (the head size): a KV-cache word holds one head at one position, so a score and a weighted-value accumulation take one cycle per cached position; a head costs $2(t+1)$ cycles plus one reciprocal. Divisions use a restoring divider that starts at bit Q_B when the quotient is known to be $< 2^{Q_B}$ (41 instead of 64 cycles for normalization, 18 instead of 34 for the softmax reciprocal). Element-wise units read up to three INT8 vectors per cycle.

TABLE III
Engine operations.

Operation	Semantics
`EMB16/EMB8/EMBWIN`	embedding (+ position) / window of k embeddings
`TABLOGIT, HRESET`	bigram logits; $h \leftarrow h_0$ at $t = 0$
`NORM`	LN/RMS over segments, INT16 or INT8 in/out
`GEMV`	$W[x; x'] + b$; INT8 out, residual add, table, logits
`ROPE, KVSTORE, ATTN`	rotation; cache write; GQA softmax-average
`MUL, GRUUPD, COPY, REQ, BOW`	element-wise operations

Cycle model. Because every unit is a fixed state machine, the cycles of the token at position t follow from the microcode in closed form:

$$\begin{aligned} C(t) = {} & 3 + 2N_{\mathrm{instr}} + \textstyle\sum_{\mathrm{NORM}} S\,(2n + 73) + C_{\mathrm{other}} \\ & + \textstyle\sum_{\mathrm{GEMV}} \left[n_{\mathrm{in}} + (B-1)\max(n_{\mathrm{in}}, P{+}2) + P + 5\right] \\ & + \textstyle\sum_{\mathrm{ATTN}} H\,\big(2(t+1) + 30\big), \end{aligned} \tag{1}$$

with $B = \lceil n_{\mathrm{out}}/P\rceil$ blocks per layer, S normalization segments of length n, H heads, and small linear terms for the remaining operations (C_{other}: embeddings $N{+}2$, RoPE $2 + 3d_h/2$ per head, element-wise n or $n{+}3$). Eq. (1) reproduces the simulated cycles of every model at every position exactly (328 (run, position) pairs, maximum deviation 0 cycles), so the hardware cost of an architecture can be read off its program without simulation or synthesis.

Storage. Eleven INT8 vector buffers (P-byte words, byte-enable writes, three asynchronous read ports), the INT16 residual and logit memories, the KV cache, the running sums of the bag-of-words model and the token history are on chip. Weights, requantization parameters, tables and activation tables are block RAMs.

One bitstream. With `LOADABLE=1` the model memories are RAMs written through a load port; an AXI4-Lite wrapper exposes the load port, per-step control, a logit window, a hardware argmax and a cycle counter to the ARM cores of the Zynq. The *universal* configuration is sized for the maximum over all models ($V = 512$, $T = 256$, five layers); a Python driver loads any model in milliseconds to seconds and runs it without re-synthesis. With `SPECIALIZE=1` the units a model does not use are removed, which we use to attribute resources to architectural features.

TABLE IV
VERIFICATION EVIDENCE (ALL SIMULATIONS WITH ASSERTIONS ON ADDRESS RANGES, INT32 ACCUMULATOR OVERFLOW AND THE DIVIDER'S PRECONDITION).

Check	Result
Every trained model, test set, all logits	13 checkpoints, 2,846,072 logits, 0 mismatches
Same, specialized engine and load port	0 mismatches
One-bitstream engine via AXI (10 models)	0 mismatches; 200-token story identical
Randomized differential tests	60 random architectures/sizes/weights, 1,256,707 logits, 0 mismatches
4-state simulation (Icarus), X on outputs	0 unknown logits
Divider, exhaustive (W=12) + random	36,421,426 cases, 0 errors
Independent C implementation of the ISA	bit-exact on all models
On-board (PYNQ-Z2), all models	**TBD**

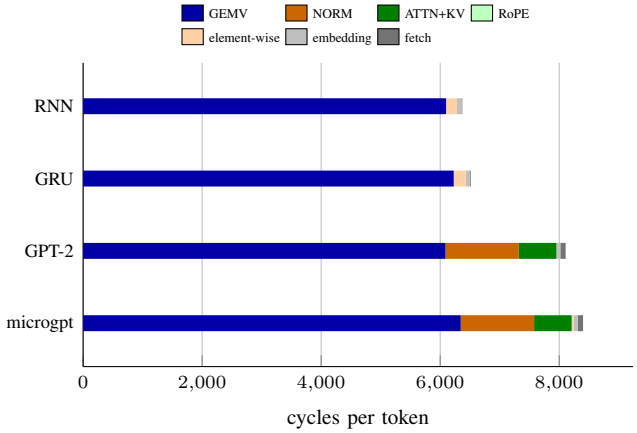


Fig. 2. Cycles per token by operation (names, 50k parameters).

VI. VERIFICATION

Every claim about the hardware rests on bit-exact equality with the integer reference. Table IV summarizes the evidence.

The simulations use Verilator 5.052; the randomized tests draw an architecture, vocabulary (27–512), context (8–64), depth (1–3) and width at random, fill the weights with values of random scale, calibrate on random tokens, and compare every logit of random token sequences. The divider's skip-ahead is proven by exhaustive simulation of every valid input at $W = 12$ and by the property $q\,b \le a < (q+1)\,b$ on random 64-bit cases; the softmax reciprocal is checked for every denominator in $[2^{15}, 2^{24})$. A third implementation, a C instruction-set simulator of the microcode, matches as well and serves as the CPU baseline.

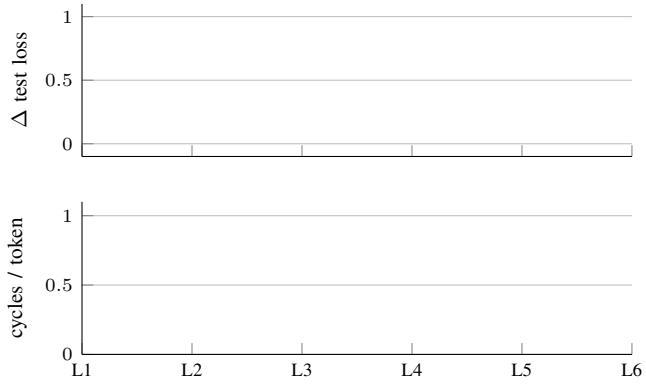


Fig. 3. Feature ladder: change of test loss with respect to L0 (top) and engine cycles per token on names (bottom).

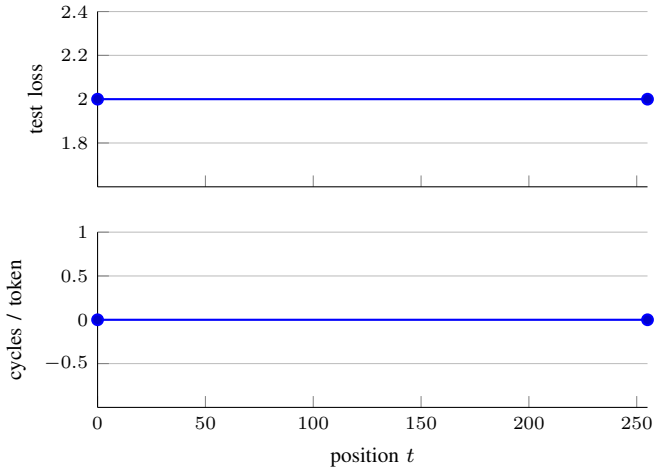


Fig. 4. TinyStories: test loss at each position of a 256-token window (top) and engine cycles for the token at that position (bottom). Attention costs grow linearly with the position; recurrent and window models are constant.

VII. RESULTS

A. The nine architectures

B. What each idea costs: the feature ladder

C. Context length: when does attention pay off?

D. Scale

E. Where the INT8 error comes from

F. Engine design space and the one-bitstream engine

G. On-board measurements

VIII. DISCUSSION AND LIMITATIONS

Limitations. The models are small (12k–260k parameters) and the conclusions about architectures are drawn at this scale; Narang et al. [16] show that architectural conclusions do not always transfer across scales, and outlier behaviour changes with scale [37]. On TinyStories every model is trained once (one seed, one learning rate) for compute reasons. All weights and the KV cache are on chip, which bounds the model size on the XC7Z020 at roughly the size of stories260K; larger

TABLE V

QUALITY AND COST AT AN EQUAL PARAMETER BUDGET. NAMES: 50K PARAMETERS, TEST LOSS (NATS/TOKEN, MEAN ± S.D. OVER SEEDS), INT8 LOSS INCREASE Δ, ENGINE CYCLES PER TOKEN. TINYSTORIES: 260K PARAMETERS, $T = 256$, TEST LOSS AND MEAN CYCLES PER TOKEN OVER THE 256 POSITIONS. RESOURCES: SPECIALIZED ENGINE FOR THE NAMES MODEL (YOSYS, XC7Z020).

	Names (short context)			TinyStories (long context)		Engine	
Model	Test loss	Δ_{INT8}	Cycles	Test loss	Cycles	LUT	DSP
Bigram	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
MLP	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
RNN	1.945	+0.0012	6,373	**TBD**	**TBD**	**TBD**	**TBD**
GRU	1.945	+0.0005	6,515	1.610	**TBD**	**TBD**	**TBD**
BoW	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
GPT-2	1.970	+0.0019	8,109	1.438	**TBD**	**TBD**	**TBD**
microgpt	1.956	+0.0019	8,402	**TBD**	**TBD**	**TBD**	**TBD**
Llama 2	1.956	**TBD**	**TBD**	1.346	**TBD**	**TBD**	**TBD**
nanochat	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**

TABLE VI

FEATURE LADDER (ONE CHANGE PER STEP, EQUAL BUDGET): TEST LOSS AND ITS CHANGE WITH RESPECT TO L0 ON NAMES (MEAN OVER SEEDS) AND TINYSTORIES, ENGINE CYCLES PER TOKEN (NAMES) AND RESOURCES OF THE SPECIALIZED ENGINE.

	Names		TinyStories		Cycles			
Step	Loss	Δ	Loss	Δ	/token	Δ	DSP	LUT
L0 GPT-2	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
L1 +RMSNorm	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
L2 +RoPE	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
L3 +SwiGLU	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
L4 +GQA	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
L5 Llama 2	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
L6 ReLU^2	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**
L7 +QK-norm	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**	**TBD**

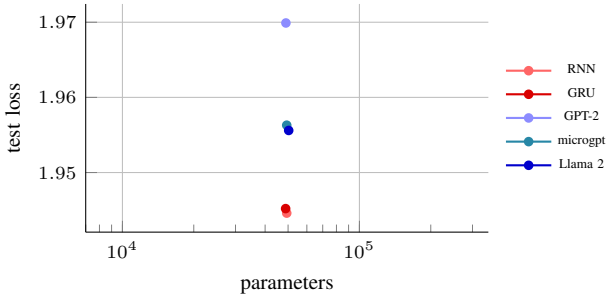


Fig. 5. Names: test loss versus parameter count (12k, 50k, 200k).

models need weights streamed from DDR, which changes the performance model from compute-bound to bandwidth-bound. The attention unit assumes a head size of 8. Resource numbers for every model come from open-source synthesis (Yosys) without timing; post-route utilization, timing and power for the one-bitstream engine come from Vivado (§VII-G). Energy is measured at the board input with a USB meter, which includes the PS and the DDR; we therefore report dynamic power above idle.

Reproducibility. The repository contains the training campaigns (seeds, learning-rate sweeps, logs), the compiler, the integer reference, the generated memories and golden vectors of every model, the RTL and testbenches, the Vivado scripts for the overlay and the out-of-context sweep, the PYNQ driver and measurement scripts, and the scripts that generate every table and figure of this paper from the raw results. A continuous-integration job re-runs the bit-exact regression on every commit.

TABLE VII

FAKE-QUANTIZATION ABLATION: LOSS INCREASE WHEN ONLY ONE TENSOR CLASS IS QUANTIZED (MAX CALIBRATION, BOUND-DERIVED NORMALIZATION SCALE).

Quantized tensors	GPT-2	Llama 2	stories260K
weights (INT8, per row)	+0.0006	−0.0010	+0.0011
q, k, v / linear outputs	+0.0002	−0.0004	+0.0094
activation tables	+0.0028	+0.0035	+0.0054
SwiGLU product	+0.0000	+0.0089	+0.0128
normalized vectors	+0.0068	+0.0037	+0.0096
residual (INT16)	−0.0001	+0.0001	−0.0000
logits (INT16)	+0.0000	−0.0000	−0.0000
all together	+0.0077	+0.0115	+0.0436

TABLE VIII

CALIBRATION OF THE INT8 PROGRAM, STORIES260K (20 TEST STORIES).

Percentile	Norm scale	INT8 loss	Δ	argmax agr. (%)
max	bound	1.4648	+0.0517	88.5
max	calibrated	1.4533	+0.0401	90.0
99.99	bound	1.4474	+0.0343	90.8
99.9	calibrated	1.4407	+0.0276	91.3
99.99	calibrated	1.4360	+0.0229	91.9
99.999	calibrated	1.4474	+0.0343	91.1

IX. CONCLUSION

REFERENCES

[1] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A neural probabilistic language model," *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003.

[2] T. Mikolov, M. Karafiát, L. Burget, J. Černocký, and S. Khudanpur, "Recurrent neural network based language model," in *Proc. Interspeech*, 2010, pp. 1045–1048.

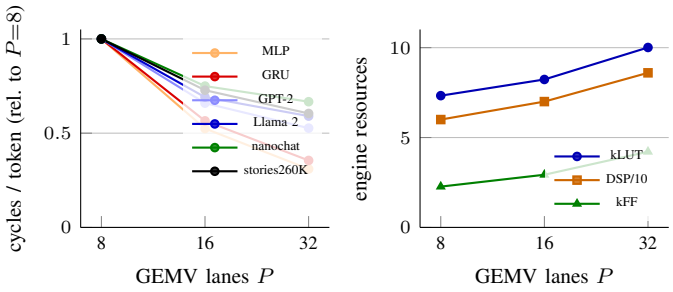


Fig. 6. GEMV lane count P: cycles per token relative to $P = 8$ (left; every point bit-exact in RTL) and engine resources (right, GPT-2 engine, Yosys). Attention and normalization keep 8 lanes, so attention-heavy models gain less.

TABLE IX
PYNQ-Z2: ENGINE LATENCY PER TOKEN, THE SAME INTEGER PROGRAM ON THE ARM CORTEX-A9, SPEED-UP AND ENERGY PER TOKEN (DYNAMIC POWER ABOVE IDLE).

Model	FPGA (µs)	ARM (µs)	Speed-up	FPGA (µJ)	ARM (µJ)
TBD (on-board measurements pending)					

[3] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[5] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," OpenAI, Tech. Rep., 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[6] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[7] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[8] N. Shazeer, "GLU variants improve transformer," arXiv:2002.05202, 2020.

[9] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 4895–4901.

[10] D. R. So, W. Mańke, H. Liu, Z. Dai, N. Shazeer, and Q. V. Le, "Searching for efficient transformers for language modeling," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 6010–6022.

[11] A. Henry, P. R. Dachapally, S. S. Pawar, and Y. Chen, "Query-key normalization for transformers," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 4246–4253.

[12] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei *et al.*, "Llama 2: Open foundation and fine-tuned chat models," arXiv:2307.09288, 2023.

[13] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, "DFX: A low-latency multi-FPGA appliance for accelerating transformer-based text generation," in *Proc. 55th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2022, pp. 616–630.

[14] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang, Y. Dai, J. Li, Z. Wang, R. Zhang, K. Wen, X. Ning, and Y. Wang, "FlightLLM: Efficient large language model inference with a complete mapping flow on FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays (FPGA)*, 2024, pp. 223–234.

[15] A. He, D. Key, M. Bulling, A. Chang, S. Shapiro, and E. Lee, "HLSTransform: Energy-efficient Llama 2 inference on FPGAs via high level synthesis," arXiv:2405.00738, 2024.

[16] S. Narang, H. W. Chung, Y. Tay, L. Fedus, T. Fevry, M. Matena, K. Malkan, N. Fiedel, N. Shazeer, Z. Lan, Y. Zhou, W. Li, N. Ding, J. Marcus, A. Roberts, and C. Raffel, "Do transformer modifications transfer across implementations and applications?" in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 5758–5773.

[17] A. Karpathy, "makemore: An autoregressive character-level language model for making more things," https://github.com/karpathy/makemore, 2022, accessed: 2026-10-13.

[18] ——, "minGPT: A minimal PyTorch re-implementation of the OpenAI GPT," https://github.com/karpathy/minGPT, 2020, accessed: 2026-10-13.

[19] ——, "nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs," https://github.com/karpathy/nanoGPT, 2022, accessed: 2026-10-13.

[20] ——, "microgpt," Blog post, https://karpathy.github.io/2026/02/12/microgpt/; code: https://gist.github.com/karpathy/8627fe009c40f57531cb18360106ce95, Feb. 2026, accessed: 2026-10-13.

[21] ——, "llama2.c: Inference Llama 2 in one file of pure C," https://github.com/karpathy/llama2.c, 2023, accessed: 2026-10-13.

[22] R. Eldan and Y. Li, "TinyStories: How small can language models be and still speak coherent English?" arXiv:2305.07759, 2023.

[23] A. Karpathy, "tinyllamas: Llama 2 architecture models trained on TinyStories (incl. stories260k)," https://huggingface.co/karpathy/tinyllamas, 2023, accessed: 2026-10-13.

[24] ——, "nanochat: The best ChatGPT that $100 can buy," https://github.com/karpathy/nanochat, 2025, accessed: 2026-10-13.

[25] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[26] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: A whitepaper," arXiv:1806.08342, 2018.

[27] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," arXiv:2004.09602, 2020.

[28] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A white paper on neural network quantization," arXiv:2106.08295, 2021.

[29] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, "I-BERT: Integer-only BERT quantization," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 5506–5518. [Online]. Available: https://proceedings.mlr.press/v139/kim21d.html

[30] Y. Lin, T. Zhang, P. Sun, Z. Li, and S. Zhou, "FQ-ViT: Post-training quantization for fully quantized vision transformer," in *Proc. 31st Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2022, pp. 1173–1179.

[31] Z. Li and Q. Gu, "I-ViT: Integer-only quantization for efficient vision transformer inference," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023, pp. 17 019–17 029.

[32] X. Hu, Y. Cheng, D. Yang, Z. Yuan, J. Yu, C. Xu, and S. Zhou, "I-LLM: Efficient integer-only inference for fully-quantized low-bit large language models," arXiv:2405.17849, 2024.

[33] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softermax: Hardware/software co-design of an efficient softmax for transformers," in *Proc. 58th ACM/IEEE Design Automation Conf. (DAC)*, 2021, pp. 469–474.

[34] J. Yu, J. Park, S. Park, M. Kim, S. Lee, D. H. Lee, and J. Choi, "NN-LUT: Neural approximation of non-linear operations for efficient transformer inference," in *Proc. 59th ACM/IEEE Design Automation Conf. (DAC)*, 2022, pp. 577–582.

[35] W. Wang, S. Zhou, W. Sun, P. Sun, and Y. Liu, "SOLE: Hardware-software co-design of softmax and LayerNorm for efficient transformer inference," in *Proc. IEEE/ACM Int. Conf. Computer Aided Design (ICCAD)*, 2023, pp. 1–9.

[36] Y. Bondarenko, M. Nagel, and T. Blankevoort, "Understanding and overcoming the challenges of efficient transformer quantization," in *Proc.*

Conf. Empirical Methods in Natural Language Processing (EMNLP), 2021, pp. 7947–7969.

[37] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "GPT3.int8(): 8-bit matrix multiplication for transformers at scale," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 30 318–30 332, arXiv:2208.07339 (as "LLM.int8()").

[38] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 38 087–38 099. [Online]. Available: https://proceedings.mlr.press/v202/xiao23c.html

[39] S. Migacz, "8-bit inference with TensorRT," NVIDIA GPU Technology Conference (GTC), San Jose, CA, presentation S7310, May 2017. [Online]. Available: http://web.archive.org/web/20240728112810/https://on-demand.gputechconf.com/gtc/2017/presentation/s7310-8-bit-inference-with-tensorrt.pdf

[40] J. L. McKinstry, S. K. Esser, R. Appuswamy, D. Bablani, J. V. Arthur, I. B. Yildiz, and D. S. Modha, "Discovering low-precision networks close to full-precision networks for efficient inference," in *Proc. 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing – NeurIPS Edition (EMC2-NIPS)*, 2019, pp. 6–9.

[41] T. J. Ham, S. J. Jung, S. Kim, Y. H. Oh, Y. Park, Y. Song, J.-H. Park, S. Lee, K. Park, J. W. Lee, and D.-K. Jeong, "A^3: Accelerating attention mechanisms in neural networks with approximation," in *Proc. IEEE Int. Symp. High Performance Computer Architecture (HPCA)*, 2020, pp. 328–341.

[42] H. Wang, Z. Zhang, and S. Han, "SpAtten: Efficient sparse attention architecture with cascade token and head pruning," in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2021, pp. 97–110.

[43] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "ELSA: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *Proc. ACM/IEEE 48th Annu. Int. Symp. Computer Architecture (ISCA)*, 2021, pp. 692–705.

[44] B. Li, S. Pandey, H. Fang, Y. Lyv, J. Li, J. Chen, M. Xie, L. Wan, H. Liu, and C. Ding, "FTRANS: Energy-efficient acceleration of transformers using FPGA," in *Proc. ACM/IEEE Int. Symp. Low Power Electronics and Design (ISLPED)*, 2020, pp. 175–180.

[45] Z. Li, M. Sun, A. Lu, H. Ma, G. Yuan, Y. Xie, H. Tang, Y. Li, M. Leeser, Z. Wang, X. Lin, and Z. Fang, "Auto-ViT-Acc: An FPGA-aware automatic acceleration framework for vision transformer with mixed-scheme quantization," in *Proc. 32nd Int. Conf. Field-Programmable Logic and Applications (FPL)*, 2022, pp. 109–116.

[46] H. Chen, J. Zhang, Y. Du, S. Xiang, Z. Yue, N. Zhang, Y. Cai, and Z. Zhang, "Understanding the potential of FPGA-based spatial acceleration for large language model inference," *ACM Trans. Reconfigurable Technology and Systems*, vol. 18, no. 1, pp. 1–29, 2025.

[47] H. Xu, Y. Li, and S. Ji, "LlamaF: An efficient Llama2 architecture accelerator on embedded FPGAs," in *Proc. IEEE 10th World Forum on Internet of Things (WF-IoT)*, 2024, pp. 1–7.

[48] Y. Qiao, Z. Chen, Y. Zhang, Y. Wang, and S. Huang, "TeLLMe: An efficient end-to-end ternary LLM prefill and decode accelerator with table-lookup matmul on edge FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field Programmable Gate Arrays (FPGA)*, 2026, pp. 247–257.

[49] J. Haris, R. Saha, W. Hu, and J. Cano, "Designing efficient LLM accelerators for edge devices," arXiv:2408.00462, 2024.

[50] Y. Hao and S. Quigley, "The implementation of a deep recurrent neural network language model on a Xilinx FPGA," arXiv:1710.10296, 2017.

[51] S. Han, J. Kang, H. Mao, Y. Hu, X. Li, Y. Li, D. Xie, H. Luo, S. Yao, Y. Wang, H. Yang, and W. J. Dally, "ESE: Efficient speech recognition engine with sparse LSTM on FPGA," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2017, pp. 75–84.

[52] S. Wang, Z. Li, C. Ding, B. Yuan, Q. Qiu, Y. Wang, and Y. Liang, "C-LSTM: Enabling efficient LSTM using structured compression techniques on FPGAs," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2018, pp. 11–20.

[53] K. Guo, L. Sui, J. Qiu, J. Yu, J. Wang, S. Yao, S. Han, Y. Wang, and H. Yang, "Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA," *IEEE Trans. Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 1, pp. 35–47, 2018.

[54] J. Fowers, K. Ovtcharov, M. Papamichael, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, L. Adams, M. Ghandi, S. Heil, P. Patel, A. Sapek, G. Weisz, L. Woods, S. Lanka, S. K. Reinhardt, A. M. Caulfield, E. S. Chung, and D. Burger, "A configurable cloud-scale DNN processor for real-time AI," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Computer Architecture (ISCA)*, 2018, pp. 1–14.

[55] Y. Yu, C. Wu, T. Zhao, K. Wang, and L. He, "OPU: An FPGA-based overlay processor for convolutional neural networks," *IEEE Trans. Very Large Scale Integration (VLSI) Systems*, vol. 28, no. 1, pp. 35–47, 2020.

[56] T. Moreau, T. Chen, L. Vega, J. Roesch, E. Yan, L. Zheng, J. Fromm, Z. Jiang, L. Ceze, C. Guestrin, and A. Krishnamurthy, "A hardware–software blueprint for flexible deep learning specialization," *IEEE Micro*, vol. 39, no. 5, pp. 8–16, 2019.

[57] H. Khan, A. Khan, Z. Khan, L. B. Huang, K. Wang, and L. He, "NPE: An FPGA-based overlay processor for natural language processing," in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2021, p. 227, extended version: arXiv:2104.06535.

[58] H. Wang, Z. Wu, Z. Liu, H. Cai, L. Zhu, C. Gan, and S. Han, "HAT: Hardware-aware transformers for efficient natural language processing," in *Proc. 58th Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 7675–7688.

[59] S. Kim, C. Hooper, T. Wattanawong, M. Kang, R. Yan, H. Genc, G. Dinh, Q. Huang, K. Keutzer, M. W. Mahoney, Y. S. Shao, and A. Gholami, "Full stack optimization of transformer inference: a survey," arXiv:2302.14017; presented at the ASSYST Workshop at ISCA 2023, 2023.